

DOCUMENTAÇÃO DE PROJETO

Amazon EC2 — Troubleshooting

Diagnóstico e Correção de Dois Bugs Reais em Script CLI de Implantação de Pilha LAMP

Parâmetro	Detalhe
Serviço principal	Amazon EC2 + AWS CLI
Foco técnico	Troubleshooting — diagnóstico e correção de dois problemas reais
Ambiente implantado	Pilha LAMP (Linux, Apache, MariaDB, PHP) — aplicação de cafeteria
Duração aproximada	45 minutos

1. Problema / Contexto

Implantações automatizadas via script são o padrão em ambientes de produção — mas scripts com erros não geram alertas visuais antes de falhar. Quando um pipeline de infraestrutura quebra, o engenheiro precisa saber ler a mensagem de erro, localizar a linha responsável no código e corrigir a causa raiz sem introduzir novos problemas.

Neste projeto, a tarefa não era criar recursos do zero: era operar sobre um script de shell existente, propositalmente danificado com dois bugs, responsável por implantar uma pilha LAMP completa no Amazon EC2. A aplicação hospedada — uma loja de cafeteria com menu, carrinho de compras e histórico de pedidos persistido em banco de dados MariaDB — só funcionaria se ambos os problemas fossem identificados e corrigidos.

O ambiente de execução era uma instância EC2 pré-provisionada (CLI Host) com a AWS CLI instalada, acessada via EC2 Instance Connect. Todo o diagnóstico, edição de script e validação foi feito a partir desse terminal.

A pilha LAMP implantada pelo script tinha a seguinte composição:

Letra	Componente	Função
L	Linux (Amazon Linux 2)	Sistema operacional da instância EC2
A	Apache (httpd)	Servidor web responsável por servir as páginas HTTP
M	MariaDB	Banco de dados relacional para persistência dos pedidos
P	PHP	Linguagem de script server-side da aplicação da cafeteria

2. Objetivo

Meu objetivo técnico foi executar um ciclo completo de troubleshooting — da falha à validação — sobre um script de infraestrutura real. De forma específica:

- Analisar o script `create-lamp-instance-v2.sh`, compreender sua estrutura completa e identificar os dois erros intencionais sem auxílio externo.
- Diagnosticar e corrigir o Problema 1: falha de execução com erro `InvalidAMIID.NotFound` causado por inconsistência de região no script.
- Diagnosticar e corrigir o Problema 2: aplicação web inacessível por bloqueio de porta no grupo de segurança, utilizando `nmap` para confirmar o diagnóstico.
- Validar a pilha LAMP resultante de ponta a ponta — do log de instalação até as funcionalidades da aplicação da cafeteria.

3. Solução

A solução foi estruturada como um ciclo clássico de troubleshooting: leitura do erro, hipótese de causa raiz, correção cirúrgica e validação. Cada problema foi tratado de forma independente e sequencial, sem modificar partes do script não relacionadas ao bug diagnosticado.

3.1 Preparação — Análise do Script e Backup

Antes de qualquer alteração, criei um backup do script original. Esse é o primeiro passo em qualquer edição de arquivo de configuração ou automação — sem backup, uma edição incorreta pode destruir o ponto de partida sem possibilidade de reversão.

```
# Navegar ao diretório do script
cd ~/sysops-activity-files/starters

# Criar backup antes de qualquer edição
cp create-lamp-instance-v2.sh create-lamp-instance.backup

# Analisar o script com numeração de linhas ativa
view create-lamp-instance-v2.sh
:set number
```

O mapeamento completo do script antes de qualquer execução:

Linhas	O que o trecho executa
1	Shebang: <code>#!/bin/bash</code>
7–11	Define o tipo de instância como <code>t3.small</code>
16–29	Pesquisa em todas as regiões AWS por uma VPC chamada "VPC da cafeteria" e captura o ID e a região
31–55	Busca dinamicamente: ID da sub-rede, nome do par de chaves e ID da AMI mais recente do Amazon Linux 2
57–122	Limpeza: verifica se já existem instância <code>cafeserver</code> ou grupo de

	segurança cafeSG e solicita exclusão
124–152	Cria o grupo de segurança com as regras de entrada necessárias
154–168	Executa o run-instances com todos os parâmetros coletados e o script de User Data
179–188	Loop que aguarda a atribuição de IP público e o exibe no terminal

3.2 Problema 1 — AMI ID Inválido por Inconsistência de Região

Na primeira execução do script, a falha ocorreu de forma imediata — antes mesmo de qualquer recurso ser criado. A mensagem de erro foi direta o suficiente para apontar o componente responsável:

```
# Erro retornado na primeira execução do script
An error occurred (InvalidAMIID.NotFound) when calling the RunInstances
operation:
The image id '[ami-xxxxxxxxxx]' does not exist
```

Sintoma

Erro InvalidAMIID.NotFound ao chamar RunInstances. A AMI referenciada não existia na região de destino.

Causa raiz e correção

A variável de região usada nas linhas 31–55 para buscar o ID da AMI não coincidia com a variável usada nas linhas 154–168 para executar o run-instances. IDs de AMI são específicos por região — o mesmo ID não existe em outra. A correção foi unificar a variável de região em todo o script.

Por que IDs de AMI são regionais: Cada região AWS mantém seu próprio catálogo de AMIs. O mesmo sistema operacional tem IDs completamente distintos em us-east-1, us-west-2, sa-east-1 e em qualquer outra região. Scripts robustos sempre derivam o ID da AMI dinamicamente para a região de destino — nunca fixam um ID obtido em outra região.

3.3 Problema 2 — Porta HTTP Bloqueada no Grupo de Segurança

Após corrigir o Problema 1 e re-executar o script com sucesso, a instância foi criada e recebeu IP público. Ao tentar acessar o servidor web no navegador, a página não carregou. A instância estava running — o problema estava em outra camada.

Utilizei o nmap para fazer o diagnóstico de conectividade diretamente nas portas da instância, sem especulação:

```
# Instalar o nmap na instância CLI Host
sudo yum install -y nmap

# Verificar o estado de cada porta na instância LAMP
nmap -Pn <IP-PUBLICO-DA-INSTANCIA-LAMP>
```

```
# Saída retornada — porta 80 bloqueada pelo grupo de segurança
22/tcp    open      ssh
80/tcp    filtered http
```

Sintoma

http://<IP-PÚBLICO> retornava timeout no navegador. nmap confirmou: porta 80 com status filtered — tráfego bloqueado por firewall.

Causa raiz e correção

O script criava o grupo de segurança com regra apenas para a porta 22 (SSH). A regra de entrada para TCP 80 (HTTP) estava ausente. A correção foi adicionar a regra inbound TCP 80 / 0.0.0.0/0 no trecho do script responsável pela criação do grupo de segurança (linhas 124–152).

Após a correção e re-execução do script, o nmap confirmou a mudança de estado:

```
# Saída do nmap após correção do grupo de segurança
22/tcp    open      ssh
80/tcp    open      http
```

nmap como ferramenta de diagnóstico de conectividade: O status filtered de uma porta indica que um firewall está descartando os pacotes — na AWS, isso aponta diretamente para o grupo de segurança ou para uma ACL de rede. O status closed significa que a porta é alcançável mas não há serviço escutando. Essa distinção elimina ambiguidade e direciona o diagnóstico para a camada correta imediatamente.

3.4 Validação Final — Pilha LAMP e Aplicação da Cafeteria

Com os dois problemas resolvidos, conectei à nova instância LAMP via EC2 Instance Connect e validei a instalação completa pelo log do cloud-init antes de testar a aplicação:

```
# Acompanhar o log do User Data em tempo real
sudo tail -f /var/log/cloud-init-output.log

# Ou visualizar o log completo
sudo cat /var/log/cloud-init-output.log
```

O log confirmou: instalação do MariaDB e PHP sem erros, download e extração dos arquivos da aplicação e mensagem final de conclusão da criação do banco de dados. Com a pilha validada pelo log, testei as funcionalidades da aplicação da cafeteria no navegador — menu, realização de pedidos e histórico persistido no MariaDB — todas operando corretamente.

Resumo dos dois problemas diagnosticados e corrigidos:

#	Sintoma	Causa raiz	Correção aplicada
1	InvalidAMIID.NotFound no run-instances	Variável de região inconsistente entre busca da AMI e criação da	Unificar a variável de região em todo o script

		instância	
2	Página web inacessível — porta 80 filtered no nmap	Grupo de segurança sem regra de entrada para TCP 80	Adicionar regra inbound TCP 80 / 0.0.0.0/0 no script

4. Ferramentas

Amazon EC2	AWS CLI	EC2 Instance Connect
Linux / Bash	Shell Script (análise e edição)	nmap
Apache HTTP Server (httpd)	MariaDB	PHP
cloud-init	User Data (bootstrapping)	Amazon CloudWatch

5. Resultado

Os dois bugs foram identificados, diagnosticados e corrigidos com sucesso. O script, após as correções, executou de ponta a ponta sem erros — criando a instância, configurando o grupo de segurança corretamente e implantando a pilha LAMP completa via User Data.

A validação foi realizada em duas camadas: pelo log do cloud-init, que confirmou a execução bem-sucedida de cada etapa do User Data; e pelo navegador, que retornou as páginas da aplicação da cafeteria com menu carregado, pedidos realizados e histórico persistido corretamente no banco de dados MariaDB.

O diferencial deste projeto não foi a complexidade da infraestrutura criada — foi a capacidade de operar sobre um sistema com falhas, diagnosticar as causas raiz de forma metodológica e aplicar correções cirúrgicas sem comprometer o restante do script.

6. Aprendizados

Troubleshooting real não é sobre conhecer a resposta certa — é sobre ter um método para chegar a ela quando a resposta não é óbvia. Os dois problemas deste projeto ensinaram coisas distintas sobre como sistemas falham e como diagnosticá-los com eficiência.

6.1 Ler o erro antes de tentar qualquer coisa

O Problema 1 foi resolvido em menos tempo do que o esperado porque a mensagem de erro — InvalidAMIID.NotFound — já continha a informação necessária para identificar a causa: o ID da AMI não existia. O passo seguinte foi perguntar por quê: não porque o ID estava errado em si, mas porque estava sendo buscado em uma região diferente da região de destino. Ler a mensagem completa,

resistir ao impulso de agir imediatamente e raciocinar sobre o que o erro está dizendo é o que separa um diagnóstico de cinco minutos de um diagnóstico de cinco horas.

6.2 Ferramentas de diagnóstico eliminam especulação

Quando a página web não carregou após o Problema 1 ser resolvido, havia múltiplas hipóteses possíveis: o Apache não instalou, o User Data falhou, o IP público estava errado ou o grupo de segurança estava bloqueando. Sem uma ferramenta de diagnóstico, cada hipótese exigiria uma ação — e uma ação errada pode introduzir novos problemas.

O nmap resolveu a ambiguidade em segundos: a porta 80 estava filtered, não closed. Isso eliminou todas as hipóteses de problema interno na instância e apontou diretamente para o grupo de segurança. A correção foi imediata porque o diagnóstico foi preciso.

Distinção crítica do nmap: filtered = firewall descartando pacotes (grupo de segurança ou ACL). closed = porta alcançável, sem serviço escutando. open = porta alcançável e serviço respondendo. Saber interpretar esses três estados direciona o diagnóstico para a camada correta sem tentativa e erro.

6.3 Correções cirúrgicas preservam a integridade do sistema

Em ambos os problemas, a correção foi mínima e específica: ajustar a variável de região inconsistente no Problema 1, e adicionar a regra de entrada ausente no Problema 2. Nenhuma linha além do necessário foi alterada. Isso é especialmente importante em scripts de infraestrutura — uma correção ampla que modifica partes não relacionadas ao bug pode corrigir um problema e introduzir outro sem que a conexão seja imediatamente percebida.

6.4 O backup é a primeira linha de defesa antes de qualquer edição

Criar o backup com `cp create-lamp-instance-v2.sh create-lamp-instance.backup` antes de qualquer modificação garantiu que, independentemente de qualquer erro de edição, o ponto de partida estava preservado. Em ambientes de produção, essa prática se estende a snapshots de volumes EBS antes de atualizações, versionamento de arquivos de configuração no Git e políticas de backup antes de migrações de banco de dados. O backup não é burocracia — é o que torna uma edição reversível.

6.5 O log do cloud-init é a fonte de verdade do User Data

Verificar `/var/log/cloud-init-output.log` foi o passo que fechou o ciclo de validação. Uma instância pode estar running e com a porta 80 aberta e ainda assim não servir a aplicação se o User Data falhou silenciosamente durante a instalação. O log do cloud-init registra cada comando executado, cada erro e o resultado final — é a diferença entre confirmar que a pilha LAMP está instalada e apenas supor que está.